Metode Avansate de Programare, sectia Informatică 2026–2027, LABORATOR 3

DEADLINE: Săptămâna 5

Precondiții:

- Laboratorul 3 continuă proiectul **Rețea de socializare** (vezi fișierul `Enunt Proiect Lab - Retea de socializare.pdf`) și pornește de la soluția laboratorului 2.
- Materiale: cursul 3 (excepții, fluxuri de intrare/ieșire), cerințele hands-on de la cursul 3 (`AbstractFileRepository`, `UtilizatorFileRepository`, `RepositoryFactory`) și seminarul corespunzător.

A. Cerințe funcționale:

- Nu sunt cerințe funcționale noi pentru acest laborator.
- Funcționalitățile din laboratorul 2 (add/remove utilizator, add/remove prieten, numărul de comunități, cea mai sociabilă comunitate) trebuie să funcționeze în continuare, de data aceasta cu datele păstrate în fișier.

B. Cerințe non-funcționale (vezi cursul 3): 10p

1. Persistența datelor în fișier text (3.5p)

- Utilizatorii și prieteniile se păstrează în fișiere text separate, în folderul `data/` al proiectului, câte o entitate pe linie, cu câmpurile separate prin `;` (0.5p):
  - `data/utilizatori.txt`: `id;prenume;nume`
  - `data/prietenii.txt`: `idUtilizator1;idUtilizator2;dataPrieteniei` (data în format ISO, de exemplu `2026-10-19T10:30`)
- Clasa abstractă generică `AbstractFileRepository<ID, E extends Entity<ID>>` extinde `InMemoryRepository<ID, E>`, încarcă datele la pornire și rescrie fișierul după fiecare `save`, `delete` sau `update` reușit. Subclasele (`UtilizatorFileRepository`, `PrietenieFileRepository`) implementează doar conversia linie ↔ entitate (`extractEntity`, `createEntityAsString`), după șablonul Template Method. Codul de citire și scriere nu se duplică în subclase. (1.5p)
- Lucrul cu fișierele: fluxuri pe caractere cu decoratori (`BufferedReader` peste `FileReader`, `BufferedWriter` peste `FileWriter`) sau NIO.2 (`Path`, `Files.newBufferedReader`, `Files.newBufferedWriter`). Toate resursele se deschid în try-with-resources. (0.5p)
- Datele încărcate din fișier trec prin validator. O prietenie se poate salva doar între utilizatori existenți, iar ștergerea unui utilizator șterge și prieteniile lui: după orice operație, cele două fișiere rămân consistente. (1p)

2. Șablonul de proiectare Factory (2.5p)

Scopul: la laboratorul 4, persistența în fișier va fi înlocuită cu persistența în baza de date, fără a modifica service-ul și interfața cu utilizatorul. **Atenție: repository-urile pentru baza de date NU se implementează la acest laborator.** Se pregătește doar terenul: factory-ul, configurarea și clasele schelet, astfel încât la laboratorul 4 să se scrie doar codul JDBC din metodele acestor clase.

- Clasa `RepositoryFactory` (Simple Factory) este singurul loc din aplicație care cunoaște clasele concrete de repository. Are câte o metodă de creare pentru fiecare repository (`createUtilizatorRepository(validator)`, `createPrietenieRepository(validator)`), care întoarce tipul abstract `Repository<ID, E>`. Ambele repository-uri se creează pentru același tip de persistență. (0.75p)
- Tipul de persistență (enum-ul `RepositoryType { MEMORY, FILE, DATABASE }`) și parametrii lui (căile fișierelor, `db.url`, `db.user`, `db.password`) se citesc din `src/main/resources/config.properties`, încărcat cu `java.util.Properties` într-un try-with-resources; nu sunt scrise în cod. O valoare lipsă sau necunoscută produce o excepție cu mesaj clar. (0.75p)
- Pentru `DATABASE`, clasele `UtilizatorDbRepository` și `PrietenieDbRepository` există **doar ca schelet**: implementează `Repository` și primesc prin constructor validatorul și parametrii conexiunii (`db.url`, `db.user`, `db.password`), dar **metodele nu se implementează**, ci aruncă `UnsupportedOperationException`. Factory-ul le creează deja pentru `repository.type=DATABASE`. Implementarea lor (JDBC) este cerința laboratorului 4. (0.25p)
- Service-ul, UI-ul și `Main` depind doar de `Repository` și nu conțin `new` pe repository-uri concrete. Criteriu de verificare: la laboratorul 4 se implementează repository-urile pentru baza de date și se schimbă `repository.type=DATABASE` în configurare; service-ul, UI-ul și `Main` rămân neschimbate. (0.75p)

3. Tratarea excepțiilor (2p)

- Excepții proprii, unchecked: `ValidationException` (laboratorul 2) și o excepție nouă pentru erorile de persistență (de exemplu `RepositoryException`), care împachetează excepția originală (`IOException`, `NumberFormatException`, `DateTimeParseException`) prin constructorul `(String message, Throwable cause)` (exception chaining). (0.75p)
- Fișier inexistent: aplicația pornește cu repository-ul gol, iar fișierul se creează la prima salvare. Linie coruptă sau invalidă în fișier: aplicația nu se oprește brusc; linia este raportată (numărul liniei și motivul) și ignorată, sau încărcarea se oprește cu un mesaj clar. Alegerea se justifică în `readme.txt`. (0.5p)
- Excepțiile se prind acolo unde pot fi tratate: în UI, unde se afișează un mesaj pentru utilizator, nu în repository sau service. Nu se acceptă blocuri `catch` goale, `e.printStackTrace()` ca singură tratare și nici `catch (Exception e)` / `catch (Throwable t)` generic. (0.75p)

4. Documentație și contracte (1p)

- Javadoc în engleză pentru interfețe și metodele publice (`@param`, `@return`, `@throws`), inclusiv pentru excepțiile unchecked. Excepțiile unchecked se documentează cu `@throws`, dar nu se declară în clauza `throws` din antetul metodei. (0.5p)
- Implementările respectă contractul din Javadoc-ul interfeței `Repository` (de exemplu, `delete(null)` aruncă `IllegalArgumentException`). `Entity` și `Tuple` suprascriu corect `equals` și `hashCode`. (0.5p)

5. Demonstrație la predare (1p)

- Se arată: adăugări și ștergeri, închiderea și repornirea aplicației cu datele păstrate, comportamentul la un fișier lipsă și la o linie coruptă introdusă manual în fișier, schimbarea `repository.type` în `MEMORY` și în `DATABASE`. (1p)

C. **Restricții de implementare:**

- **Nu se implementează persistența în baza de date** (JDBC, SQL, driver în `build.gradle`): este cerința laboratorului 4. La acest laborator, repository-urile DB sunt doar schelete care aruncă `UnsupportedOperationException`.
- Nu se folosesc interfețe funcționale, expresii lambda, method references și operații pe stream-uri (Java 8, cursul următor): citiți cu `BufferedReader.readLine()` sau `Files.readAllLines(...)` și parcurgeți cu `for`.
- `switch`-ul se scrie în forma clasică (`case X: ... return ...;`), fără `->`.
- Nu se folosește serializarea binară (`ObjectOutputStream`): fișierele sunt text, lizibile și editabile manual. Nu se folosesc biblioteci externe pentru fișiere (CSV, JSON etc.).
- Căi relative (`data/...`), fără căi absolute de tipul `C:\Users\...`. Fișierele folosesc codificarea UTF-8.
- Pentru trecerea de la persistența în memorie la cea în fișier nu se modifică clasele din pachetele `domain` și `service`; se modifică doar pachetul `repository`, factory-ul și fișierul de configurare.

D. **Sugestii și observații:**

- Porniți de la codul hands-on de la cursul 3. Atenție la:
  - la încărcarea din fișier apelați `super.save(...)`, nu `save(...)`: varianta suprascrisă ar rescrie fișierul chiar în timp ce îl citiți;
  - nu apelați din constructor metode care pot fi suprascrise și folosesc câmpuri ale subclasei: câmpurile subclasei nu sunt încă inițializate (Effective Java, Item 19);
  - `linie.split(";", -1)` păstrează câmpurile goale de la finalul liniei; `split` primește o expresie regulată (pentru `|` se scrie `"\\|"`);
  - prietenia (1, 2) și prietenia (2, 1) sunt aceeași prietenie: normalizați id-ul (de exemplu, id-ul mai mic primul), altfel `findOne` nu o găsește;
  - `equals` din `Tuple` trebuie să trateze `null` și obiecte de alt tip (fără cast direct).
- Un Simple Factory cu `switch` trebuie modificat atunci când apare un tip nou de persistență. Alternativa este o interfață `RepositoryFactory` cu câte o implementare pentru fiecare tip (`FileRepositoryFactory`, `DbRepositoryFactory`), adică Abstract Factory. Discutați în `readme.txt` avantajele și dezavantajele celor două variante din punctul de vedere al principiului OCP.
- Opțional, factory-ul poate fi implementat ca Singleton (cursul 3).
- Gândiți-vă unde puneți validarea (repository sau service) și argumentați alegerea; vezi discuția de la curs.

E. **Livrabile:**

- Codul sursă (proiect Gradle / IntelliJ), împreună cu `config.properties`, folderul `data/` și un fișier de test care conține linii corupte.

- Documentul `readme.txt`, în care precizați: cum funcționează factory-ul și ce trebuie schimbat pentru trecerea la baza de date; cum tratați fișierul lipsă și liniile corupte; ce principii de proiectare (SOLID: DIP, OCP, SRP, LSP) și ce șabloane (Factory, Template Method, Strategy, Decorator) se regăsesc în proiect și unde.
- Laboratorul se încarcă în assignment-ul de pe Teams. Dacă laboratorul nu este încărcat în assignment, NU se consideră predat.